Credit Risk Modeling with Cost-Sensitive Decision Making

Which credit decision maximizes expected bank profit? | Kaggle Credit Risk Dataset

32,576
CLEANED DATASET
76% of interest-rate data was corrupted at source; handled via grade-level median imputation

21.82%
OVERALL DEFAULT RATE
Matches known benchmark for this dataset after bias-checked cleaning

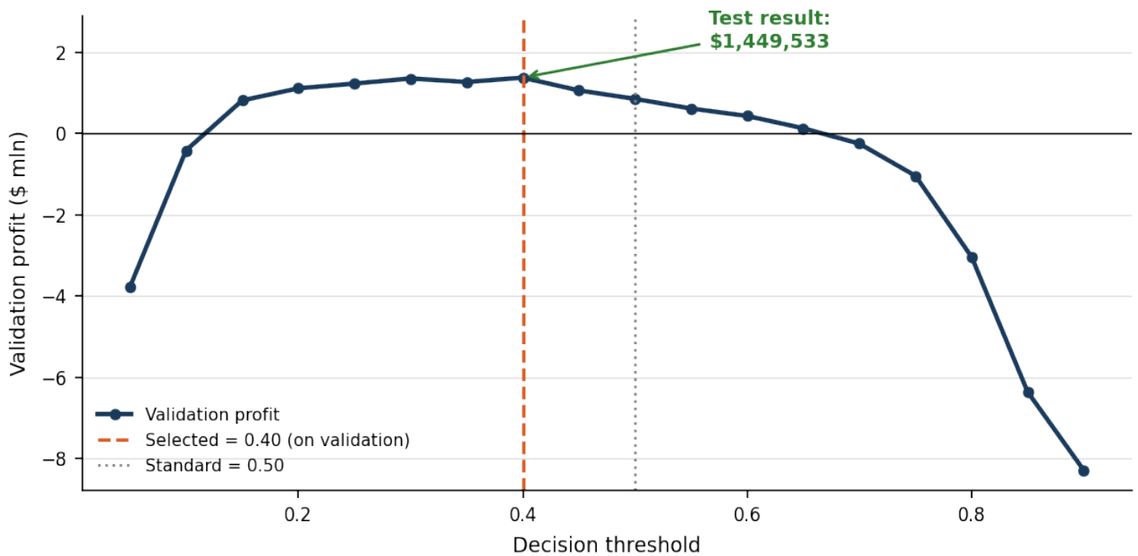
AUC 0.912
MODEL QUALITY (TEST)
Random Forest; threshold selected on a separate validation set

Threshold 0.40
OPTIMAL DECISION POINT
Selected on validation only, then confirmed once on untouched test data

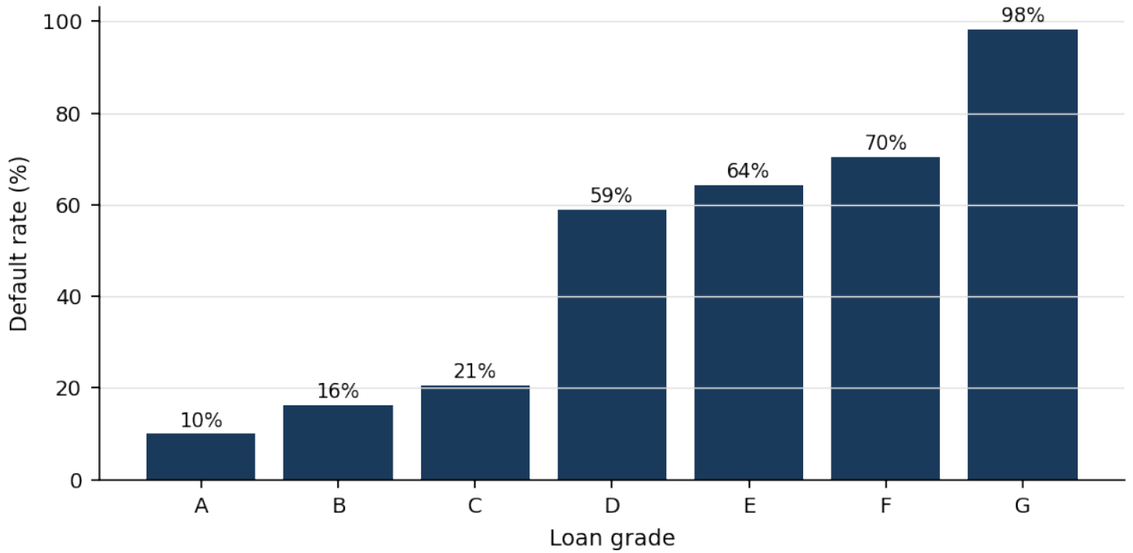
+$323K
GAIN FROM THRESHOLD TUNING
vs. standard 0.50; honest out-of-sample estimate on test set

-16%
COST-SENSITIVE TRAINING
Underperformed simple threshold tuning -- see page 3

Threshold Selection (Validation) & Honest Test Result



Default Rate by Loan Grade



Key read

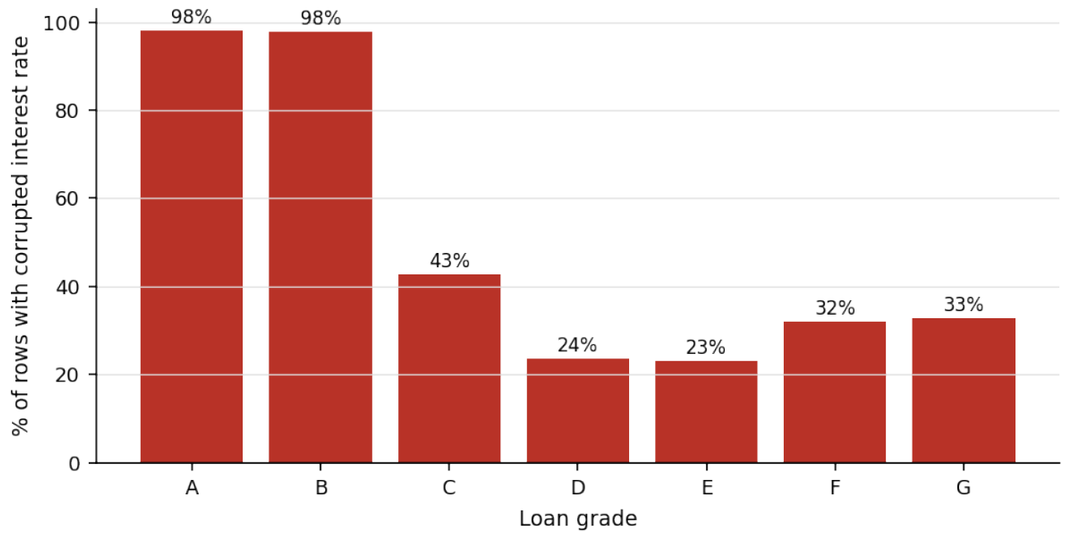
The profit-optimal threshold (0.40) was selected using a validation set the model never influenced decisions with, then applied exactly once to an untouched test set -- avoiding the classic mistake of tuning a decision rule on the same data used to report its performance. That honest test result: $1,449,533, a $323,413 improvement over the standard 0.50 cutoff.

Source: Kaggle Credit Risk Dataset (laotse) | Analysis: 2026-09

Data Quality: A Hidden Bias Discovery

How a routine cleaning step almost skewed the entire analysis

% of Rows with Corrupted Interest Rate, by Loan Grade



What happened

1. The problem

76% of interest-rate values were corrupted -- garbled bytes instead of numbers (e.g. "16.fev" instead of "16.02").

2. The cause

Byte-level inspection revealed the garbled text decoded to Russian month abbreviations ("fev" for February, "noya" for November, in Cyrillic). The file had been opened in Excel with a Russian locale, which auto-converted decimal numbers into dates -- destroying the original values beyond recovery. The original rates could not be recovered; they were replaced with grade-level estimates instead (step 4).

3. The near-mistake

The obvious fix -- drop corrupted rows, keep only the clean 7,811 -- was tested first. But checking WHERE the corruption concentrated revealed grade A (98.2% missing) and grade B (98.0% missing) loans -- the safest customers -- were almost entirely affected, while grade C-G ranged 24-43%. Dropping rows would have skewed the whole dataset toward risky customers.

4. The fix

Instead of discarding data, missing interest rates were imputed using the median rate for that same loan grade -- preserving every customer without introducing the bias that deletion would have caused.

5. Validation

Compared default rates for imputed vs. real interest-rate rows, WITHIN each grade. They were reasonably close (e.g. grade D: 58.9% vs 59.4%) -- evidence the imputation did not materially alter the observed default pattern within grade.

Why this matters

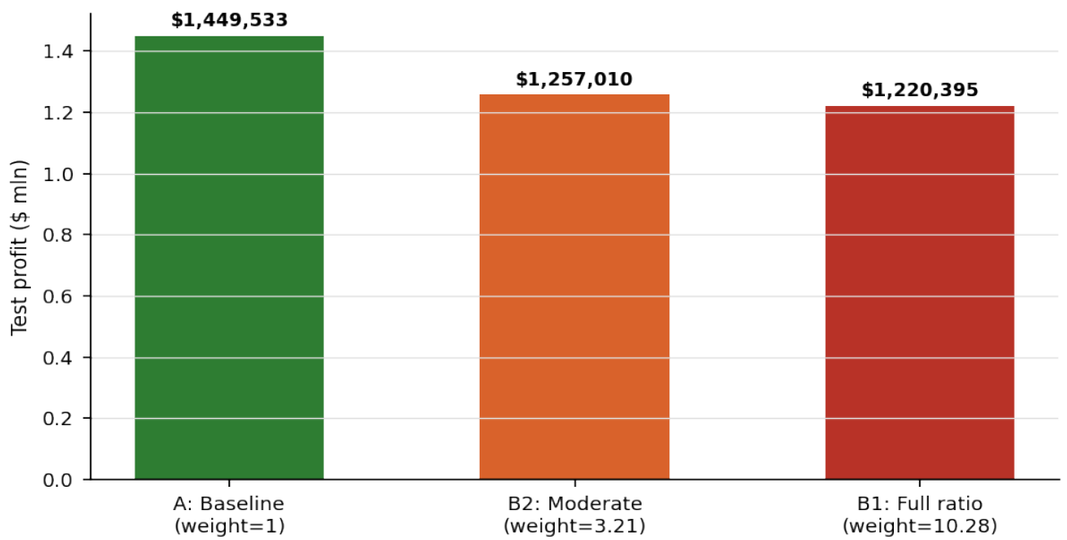
The easy cleaning shortcut (delete corrupted rows) would have quietly removed almost all of the safest customers from the analysis -- a lesson in checking WHY data is missing, not just filling gaps.

Source: Kaggle Credit Risk Dataset (laotse) | Analysis: 2026-09

Does Cost-Sensitive Training Help?

Approach A (threshold tuning) vs Approach B (cost-sensitive training)

Test Profit by Approach (honest, out-of-sample)



The experiment

Two approaches were tested for turning model predictions into approve/reject decisions. For both, the decision threshold was selected using the validation set only, then evaluated once on the untouched test set:

Approach A: Baseline + threshold tuning

Train a standard model, then search for the profit-maximizing decision threshold on validation. Result: $1,449,533 on test, at threshold 0.40.

Approach B: Cost-sensitive training

Bake the ~10.28x cost asymmetry (bad loan vs. good loan) directly into model training via class weights, then still tune the threshold the same way. Result: worse in every version tested -- $1,257,010 with a moderate weight (3.21x), $1,220,395 with the full weight (10.28x).

Why cost-sensitive training underperformed

Likely double-counting: cost information was already fully used once, in choosing the profit-optimal threshold. Injecting it again into training over-corrected the model's probability estimates, and the pattern was monotonic -- the stronger the weight, the worse the result. This held under both the original and the corrected (validation-based) evaluation methodology.

Practical takeaway

Start with the simplest profit-aware method: train a standard model, then tune the decision threshold on a validation set using EMP. Only add cost-sensitive training if it demonstrably outperforms on held-out data -- it did not here, and added complexity without added profit.

Source: Kaggle Credit Risk Dataset (laotse) | Analysis: 2026-09